

Code review — victor-variante-bac

Runde de review pe proiectul tău, cea mai nouă sus.

Un singur fișier, runda cea mai nouă sus. Rundele vechi rămân dedesubt.

Runda 4 — 2026-10-06 — sha `4c9f520`

Trei commit-uri: `95205bb` `inceput code review`, `936eea4` `finish code review`, `4c9f520` `tema`.
21 de fișiere atinse, 737 de linii adăugate. **14 variante noi** (67, 73, 80–86, 91–95) și **toate cele 5**  **din runda 3 atinse**.

Codul NU a fost modificat — totul e doar aici.

Verificat prin **compilare și rulare**: fiecare header compilat separat cu `clang++ -std=c++17 -Wall -Wextra`, apoi rulat cu `-fsanitize=address,undefined`. Fiecare variantă nouă rulată pe exemplul din enunț **și pe cazuri pe care enunțul nu le arată** (niciun număr potrivit, valori la capete, cifre care se repetă de mai mult de două ori).

Verdictul pe scurt.

Poarta de compilare: trecută. 46/46 headere. A patra rundă la rând.

Runda 3: 3 din 5  **închise curat, 1 pe jumătate, 1 rescris și tot greșit.** `doarInA` afișează `NU EXISTA` , `alKlea` are `return -1` , `sub` există și e corect pe **7 din 7** cazuri . Iar tiparul din runda 3 — „review-ul devine comentariu” — **s-a închis**: în `varianta24.h` fix-ul e acum în cod, nu în `//`.

Variantele noi: 11 din 14 corecte. Iar la două dintre ele ai evitat exact capcanele din `victor-vectori`: `% 2 != 0` pe numere negative (`varianta84`) și `abs` înainte de a lua cifrele (`varianta83`). Asta e regulă învățată, nu linie reparată.

Tiparul runde 4: exemplul din enunț e singurul tău test. Toate cele trei  de mai jos **trec** pe exemplul din enunț, sau n-au fost rulate deloc. `varianta3` dă `53211235` corect — dar exemplul are fiecare cifră de exact două ori, singurul caz în care greșeala nu se vede. `varianta80` are o ramură `NU EXISTA` care nu se poate executa niciodată — iar exemplul are numere bune, deci n-ai cum s-o vezi. `varianta73` întoarce `-1` pe orice intrare, și nu are niciun apel.

Exemplul din enunț e făcut să-ți arate **ce** se cere. Nu e făcut să-ți găsească bug-urile. Pentru asta ai nevoie de **al doilea exemplu, inventat de tine**, ales anume ca să încerce să strice programul.

Starea constatărilor din runda 3

#	fișier	stare
B1 fix-ul scris ca comentariu	varianta24.h	🟡 PE JUMĂTATE — 5.667 ✅, dar <code>mini</code> și <code>maxi</code> se afișează în continuare → M1 mai jos
B2 <code>doarInA</code> fără <code>NU EXISTA</code>	tema...doi-indici.h	✅ ÎNCHIS — <code>NU EXISTA</code> pe <code>(4,6)/(4,6,8)</code> și pe <code>(5,5)/(5,5)</code>
B3 palindromul neconstruit	varianta3.h	🔴 RESCRIS, tot greșit — exemplul iese, <code>11111111</code> nu → B1 mai jos
B4 <code>sub</code> nu există	varianta8.h	✅ ÎNCHIS — <code>sub</code> corect pe 7/7 cazuri · dar <code>solutie4</code> nu-l apelează → M2
B5 <code>alKlea</code> fără <code>return</code>	tema...doi-indici.h	✅ ÎNCHIS — <code>k = 8</code> dă <code>-1</code> , sanitizerul tace
M1 proiectul compilează 1 fișier din 32	varianteBac.cpp	🔴 agrat — acum 1 din 46 → M3
M2 <code>reuniune</code> fără <code>p = 0</code> , cu <code>cout</code>	tema...doi-indici.h:188	🔴 neatins
M3 zero <code>fstream</code>	tot repo-ul	🔴 neatins, a patra rundă → M4
M4 <code>p</code> și <code>sortare</code> distrug tabloul apelantului	varianta42.h , varianta8.h:161	🔴 neatins
M5 coliziuni de nume	tot repo-ul	🔴 agravat: 18× <code>solutie3</code> , 15× solutie → M7
M6 prompturi <code>n=</code>	varianta1/6/11.h	🔴 neatins

#	fișier	stare
M7 <code>varianta46</code> : <code>k</code> necitit	<code>varianta46.h</code>	● neatins
M8 <code>int n</code> ; necitit	<code>varianta55.h:33</code>	● neatins
C1 <code>shift</code> citește <code>x[n]</code>	<code>varianta23.h:31</code>	● neatins, a treia rundă
C2 <code>int ct = 1</code> ; nefolosit	<code>varianta24.h:60</code>	● neatins
C3 variabila <code>cif</code> în funcția <code>cif</code>	<code>varianta3.h:31</code>	● neatins
C4 marcaje pe fișiere corecte	v20, v8	● neatins, și s-au înmulțit → C2
C5, C6	—	● neatins

Ce am rulat, ca să vezi dovada

`varianta3.h`, alte numere de 8 cifre puse în locul lui `21523531`:

n	ce afiseaza	trebuia	de ce
21523531	53211235	53211235	corect (exemplul)
12272351	0	0	corect (exemplul)
99887766	98766789	98766789	corect: fiecare cifra de 2 ori
11111111	10000001	11111111	cifra 1 de 8 ori
11112222	21000012	22111122	1 si 2 de cate 4 ori
22220000	20000002	22000022	2 si 0 de cate 4 ori

`varianta73.h`, `cautare` pe șirul sortat din enunț (`x[1..11]` = `-456 -98 -8 -5 0 3 7 8 65 587 635`):

```
cautare(v = -500) -> -1      trebuia 1
cautare(v = 1)    -> -1      trebuia 6
cautare(v = 5)    -> -1      trebuia 7
cautare(v = 700)  -> AddressSanitizer: stack-buffer-overflow
                   READ of size 4 ... in cautare varianta73.h:22
```

varianta80.h :

```
{30,44,111,7,25,5} -> 44 111 7 5      trebuia 5 7 44 111
{30,25,12}          -> (nimic)      trebuia NU EXISTA
```

Restul variantelor noi, pe exemplul din enunț:

```
V67 : (12,37,43,6,71)          -> 3          corect
V67 : (0,1,2,3,4,9,25)         -> 2          corect
V81 : (123,122,423,555)        -> 305.75    corect
V82 : (123,122,423,555)        -> 677        corect
V83 : (265,717,232,312,191)    -> 1140       corect
V84 : 2 3 7 8 / 44 3 1 8       -> 4          corect
V85 : 2 3 7 8 / 44 3 1 8       -> 4          corect
V86 : 5123 8 6 12 3           -> 12 8       corect
V91 : 7354 123864 51731 570    -> 2          corect
V92 : n=9 m=3                  -> (rand gol)  vezi M6
                                33.8 60 80
                                3 -7.5 -22
V93 : n=11 -> 1 11 2 10 3 9 4 8 5 7 6          corect
V94 : (6,5,3,2,1) -> 2 · (5,5,5) -> 0 · (1,3,2) -> -1 corect
V95 : (2,-4,8,-10,12)          -> 1          corect
```

Fixurile rundei 3:

doarInA	a=(4,6)	b=(4,6,8)	-> NU EXISTA	corect
doarInA	a=(5,5)	b=(5,5)	-> NU EXISTA	corect
alKlea	k=1..7		-> 1 2 5 5 7 9 12	corect
alKlea	k=8		-> -1, fara eroare	corect
sub	a={2,4,8,10,14}	b={3,5,11}	-> 2 3 4 5 8 11 14	corect
sub	a={2,4}	b={1}	-> 1 2	corect
sub	a={2,4,6}	b={1}	-> 1 2	corect
sub	a={10,20}	b={1,3,5}	-> 1 10	corect
sub	a={2,4}	b={1,3}	-> 1 2 3 4	corect
sub	a={2,100}	b={1,3,5,7}	-> 1 2 3 100	corect
sub	a={20,22}	b={1,21,23}	-> 1 20 21 22 23	corect
V24	n=5 {1,9,4,8,5}		-> 4, 8 ,5.667	vezi M1

Warning-uri, tot repo-ul:

```

varianta24.h:5: extra tokens at end of #include directive
varianta24.h:60: unused variable 'ct'
varianta3.h:60: unused variable 'p'

```

Critice

B1 — `varianta3.h` : cifra se pune o dată, oricâte apariții are

`varianta3.h:68-75`

CUM E ACUM: pe exemplul din enunț iese `53211235`, corect. Pe `11111111` iese `10000001` — un număr care nici măcar nu are cifrele lui `n`. Pe `11112222` iese `21000012` în loc de `22111122`.

```
for (int i = 9; i >= 0; i--) {
    if (f[i] != 0) {
        nou = nou + i * (fata + spate);
        fata = fata / 10;
        spate = spate * 10;
    }
}
```

DE CE: `f[i]` ține **câte** apariții are cifra `i`. Dar `if (f[i] != 0)` pune o singură întrebare: „apare sau nu?”. Pentru fiecare cifră care apare, pui **o pereche** — una în față, una în spate — și treci la cifra următoare. Deci folosești doar două apariții din fiecare cifră, oricâte ar fi.

Și de ce exemplul trece: în `21523531` **fiecare cifră apare exact de două ori**. O pereche = două apariții = toate aparițiile. E singurul caz în care „apare” și „de câte ori apare” dau același răspuns. Când o cifră apare de 4 ori, ai nevoie de **două** perechi din ea, iar codul pune una și restul pozițiilor rămân `0`. De-aia `11111111` iese `10000001`: o pereche de `1` la capete, șase zerouri la mijloc.

E exact tiparul din `victor-vectori`: **prezență în loc de multiplicitate**. Ai frecvența calculată corect, cu `cif` — ce lipsește e s-o folosești ca număr, nu ca da/nu.

FIX: câte perechi are cifra `i`? `f[i] / 2`. Deci `if`-ul devine un `for` care pune `f[i] / 2` perechi:

```

for (int i = 9; i >= 0; i--) {
    for (int k = 0; k < f[i] / 2; k++) {
        nou = nou + i * (fata + spate);
        fata = fata / 10;
        spate = spate * 10;
    }
}

```

L-am rulat pe toate cele 6 numere din tabelul de sus — iese corect pe toate.

B2 — varianta73.h: cautare întoarce -1 pe orice intrare

varianta73.h:16-30

CUM E ACUM: pe `v = 1`, `v = 5`, `v = -500` întoarce `-1`. Pe `v = 700` (mai mare decât tot șirul) programul e oprit de sanitizer: `stack-buffer-overflow`, citire în afara tabloului, la linia 22.

```

int inf = 1;
int sup = n + 1;
while (inf <= sup) {
    ...
}
return -1;

```

DE CE: două lucruri separate.

1. Răspunsul e în `inf`, nu în `-1`. Căutarea binară clasică se oprește cu `-1` când valoarea **nu e** în șir. Aici enunțul îți garantează că `v` nu e în șir — deci nu cauți „unde este”, cauți „unde ar trebui pus”. Bucla se oprește când `inf` a trecut de `sup`. În acel moment, tot ce e la stânga lui `inf` e mai mic decât `v`, tot ce e de la `inf` încolo e mai mare. Deci `inf` **este** poziția de inserare. Funcția a calculat răspunsul corect, apoi l-a aruncat și a întors `-1`.

2. `sup = n + 1` iese din tablou. Elementele sunt pe pozițiile `1..n`. Dacă `v` e mai mare decât toate, `inf` crește până la `n + 1`, `mij` ajunge `n + 1` și se citește `x[n + 1]` — un element care nu există. E **exact** bug-ul din `victor-vectori`, `cautareBinara.h:40` (`sup = dim`, runda 7, încă deschis acolo). Aceeași regulă, alt repo: `sup` e **ultima poziție validă**, nu cea de după.

Punctele b) și c) nu sunt scrise încă — ai marcat corect fișierul cu `///?` .

FIX:

```
int cautare(int n, int x[], int v) {
    int inf = 1;
    int sup = n;
    while (inf <= sup) {
        int mij = (inf + sup) / 2;
        if (x[mij] > v) {
            sup = mij - 1;
        }
        else {
            inf = mij + 1;
        }
    }
    return inf;
}
```

Rulat: `-500 → 1` , `1 → 6` , `5 → 7` , `600 → 11` , `700 → 12` , fără nicio eroare de sanitizer. Același fix, în aceeași zi, și în `cautareBinara.h` din `victor-vectori` .

B3 — `varianta80.h` : **NU EXISTA** nu se poate afișa niciodată, iar ordinea lipsește

`varianta80.h:18-38`

CUM E ACUM: pe `{30,25,12}` (niciun număr cu cifre egale) nu se afișează nimic. Pe exemplul din enunț iese `44 111 7 5` în loc de `5 7 44 111` .

```

int vf = -1;
for (int i = 0; i < d; i++) {
    vf = 1;           // la fiecare numar
    ...
}
if (vf == -1) {      // vf nu mai e -1 dupa primul pas
    cout << "NU EXISTA" << endl;
}

```

DE CE: `vf` face două treburile deodată: „**numărul ăsta** are cifrele egale?” și „am găsit **vreun** număr bun până acum?”. Prima întrebare trebuie pusă din nou la fiecare număr — de-aia `vf = 1` la începutul buclei. A doua trebuie să țină minte peste toate numerele. Prima o șterge pe a doua: după primul pas al buclei, `vf` e `1` sau `0`, niciodată `-1`. Am verificat: pe `{30, 25, 12}`, după buclă, `vf` e `0`.

Asta e continuarea lui B2 din runda 3. Acolo steagul era scris și **necitit**. Aici e citit, dar **suprascris înainte să fie citit**. Un steag care răspunde la „a fost vreodată?” se pune înainte de buclă și în buclă **doar se ridică**, nu se resetează.

Ordinea crescătoare: enunțul o cere, iar codul afișează numerele în ordinea din fișier. Ca să le ordonezi, trebuie mai întâi să le **păstrezi** — deci un al doilea tablou. Iar numărul de elemente din el, `k`, îți răspunde singur la „am găsit vreunul?": `k == 0`. Nu mai ai nevoie de steag deloc.

FIX:

```

void solutie() {
    int a[1000] = { 30,44,111,7,25,5 };
    int d = 6;
    int b[1000];
    int k = 0;

    for (int i = 0; i < d; i++) {
        int x = a[i];
        int cif = x % 10;
        bool egale = true;
        x = x / 10;
        while (x != 0) {
            if (x % 10 != cif) {
                egale = false;
                break;
            }
            x = x / 10;
        }
        if (egale) {
            b[k] = a[i];
            k++;
        }
    }

    for (int i = 0; i < k - 1; i++) {
        for (int j = i + 1; j < k; j++) {
            if (b[i] > b[j]) {
                int aux = b[i];
                b[i] = b[j];
                b[j] = aux;
            }
        }
    }

    if (k == 0) {
        cout << "NU EXISTA";
    }

    for (int i = 0; i < k; i++) {
        cout << b[i] << " ";
    }
}

```

```
}  
  
}
```

Rulat: exemplul $\rightarrow$ 5 7 44 111 , {30,25,12} $\rightarrow$ NU EXISTA , {0,3} $\rightarrow$ 0 3 . Citirea din bac.in rămâne de pus — vezi M4.

🟡 Importante

M1 — `varianta24.h` : fix-ul a fost adăugat pe linie, nu a înlocuit-o

`varianta24.h:5` și `:78`

```
#include <iomanip>;  
...  
cout << fixed << setprecision(3) << mini << ", " << maxi  
    << " , " << medie<<endl;
```

lese `4, 8 ,5.667`. Formatul mediei e bun acum — partea grea e făcută. Dar fix-ul din runda 3 era o linie întreagă, care o **înlocuia** pe cea veche. Tu ai pus începutul ei în fața liniei vechi și ai păstrat restul. Enunțul cere pe ecran doar media.

Și `;` de după `#include` nu are ce căuta acolo: `#include` nu e o instrucțiune C++, e o comandă pentru preprocesor, care se termină la capătul rândului. Compilatorul zice `extra tokens at end of #include directive`; Visual Studio dă `C4067`.

Linia 78 trebuie să fie exact asta, și nimic în plus:

```
cout << fixed << setprecision(3) << medie;
```

M2 — `varianta8.h` : `sub` e corect, dar `solutie4` rulează tot codul vechi

`varianta8.h:153–168`

`sub` e scris, e corect pe 7 din 7 cazuri, afișează singur, are parametrii din enunț. B4 e închis. Dar `solutie4` cheamă tot `interclasare`, cu `c`, `dimC` și bucla de afișare. Deci când rulezi varianta 8, rulezi **altă funcție** decât cea pe care o predai. Dacă schimbi ceva în `sub`, nu vei vedea efectul.

Plus `sortare(a, dimA)` și `sortare(b, dimB)` pe tablouri pe care enunțul ți le dă deja sortate (M4 din runda 3). `solutie4` devine:

```
void solutie4() {
    int a[100] = { 2,4,8,10,14 };
    int b[100] = { 3,5,11 };
    sub(a, 5, b, 3);
}
```

M3 — proiectul tău compilează 1 fișier din 46

`varianteBac.cpp` include doar `varianta80.h`. Celelalte 45 de headere nu sunt compilate când apeși F7. La runda 3 era 1 din 32.

Costul concret de data asta: `varianta3.h:60` are `int p = 1;` nefolosit, iar `varianta24.h:5` are `;` după `#include`. Amândouă sunt warning-uri gratuite pe care Visual Studio ți le-ar fi arătat — dacă ar fi compilat fișierele. Se rezolvă definitiv cu M7.

M4 — datele nu vin de unde cere enunțul (a patra rundă)

`grep fstream *.h` → **nimic**, în 46 de fișiere. Iar în runda asta s-au adăugat:

- **fișier** cerut, date scrise în cod: `varianta73.h` (`sir.in` / `sir.out`), `varianta80.h` (`bac.in`), `varianta86.h` (`bac.txt`)
- **tastatură** cerută, date scrise în cod: `varianta84.h`, `varianta85.h`, `varianta91.h`, `varianta92.h`, `varianta93.h`

La bac, „citește de la tastatură” și „citește din fișierul `bac.in`” sunt puncte în barem. Un program care rezolvă doar exemplul din enunț nu primește punctele de citire. Începe cu `varianta80.h`, după ce aplici B3:

```
#include <fstream>

...

ifstream fin("bac.in");
int a[1000];
int d = 0;
while (fin >> a[d]) {
    d++;
}
fin.close();
```

`while (fin >> a[d])` citește până se termină fișierul — enunțul nu spune câte numere sunt, deci nu poți folosi un `for` până la `n`.

M5 — `varianta67.h`: `d <= aux / 2` pe numere de 9 cifre

`varianta67.h:26`

Rezultatul e corect. Dar am rulat `calcul` pe trei numere prime de 9 cifre (`999999937` , `999999929` , `999999893`), compilat cu optimizări: **1,13 secunde**. Enunțul permite 10 000 de numere — adică până la o oră. Un divizor mai mare decât `√aux` are mereu pereche unul mai mic decât `√aux` , deci dacă n-ai găsit niciunul până la `√aux` , nu există.

```
for (int d = 2; d * d <= aux; d++) {
```

Pe aceleași trei numere: `0,00` secunde de procesor.

M6 — `varianta92.h`: prima linie a ieșirii e goală

`varianta92.h:45` — `cout << endl;` înainte de prima linie. Enunțul cere cele mai mari `m` „pe prima linie a ecranului”. Ieșirea ta are pe prima linie nimic, iar numerele pe a doua. Șterge linia 45.

M7 — 15 funcții `solutie` , 18 `solutie3` (agravat)

18 <code>solutie3</code>	15 <code>solutie</code>	6 <code>solutie4</code>	4 <code>f</code>
2 <code>suma</code>	2 <code>calcul</code>	2 <code>p</code>	2 <code>sort</code>

`namespace v80 { ... }` în fiecare fișier, apoi `v80::solutie()` . De aici se rezolvă și M3: `main` poate include toate headerele deodată.

Restanțe neatinse din runda 3

M2 `reuniune` fără `p = 0` · M4 `varianta42.h` sortează tabloul apelantului · M6 prompturi `n=` în `varianta1/6/11.h` · M7 `varianta46.h` nu citește `k` · M8 `int n;` în `varianta55.h:33` . Toate la fel ca în runda 3 — niciuna nu te blochează.

Cleanups

- **C1** — `varianta23.h:31` : `i < n` citește `x[n]` . Corect: `i < n - 1` . **A treia rundă.**
 - **C2** — marcasele nu mai spun unde e lucrul nefăcut. Din cele 5 fișiere noi marcate `///??` sau `//de revazut` , **3 sunt corecte** (`varianta67` , `varianta84` , `varianta85`) și 2 au  (`varianta73` , `varianta80`). Iar `varianta3.h:6` zice tot `///?? incomplet` . Un marcaj e util doar dacă îl ștergi când termini.
 - **C3** — `varianta3.h:46` `int f[100]` pentru 10 cifre; `:60` `int p = 1;` nefolosit.
 - **C4** — `varianta24.h:60` `int ct = 1;` nefolosit (a doua rundă).
 - **C5** — `varianta86.h:18` o variabilă numită `max` , cu `using namespace std;` deasupra, peste `std::max` . Merge, dar `maxim` scapă de întrebare.
-

Before / After

#	fișier	acum	trebuie
B1	varianta3.h:70	<code>if (f[i] != 0) {</code>	<code>for (int k = 0; k < f[i] / 2; k++) {</code>
B2	varianta73.h:18	<code>int sup = n + 1;</code>	<code>int sup = n;</code>
B2	varianta73.h:29	<code>return -1;</code>	<code>return inf;</code>
B3	varianta80.h:18-38	vf resetat la fiecare număr, afișare pe loc	<code>b[k++] = a[i]</code> , sortare, <code>if (k == 0) NU EXISTA</code>
M1	varianta24.h:78	<code>cout << fixed << setprecision(3) << mini << ", " << maxi<<" , " << medie<<endl;</code>	<code>cout << fixed << setprecision(3) << medie;</code>
M2	varianta8.h:161-167	sortare ×2, interclasare , bucla <code>cout</code>	<code>sub(a, 5, b, 3);</code>
M5	varianta67.h:26	<code>d <= aux / 2</code>	<code>d * d <= aux</code>
M6	varianta92.h:45	<code>cout << endl;</code>	(șters)

✓ Ce e corect — verificat prin rulare

11 din 14 variante noi dau răspunsul corect. v67, v81, v82, v83, v84, v85, v86, v91, v93, v94, v95, plus v92 pe valori.

Patru merită spuse pe nume:

varianta84 — `b[i] % 2 != 0` pe numere care pot fi negative. Enunțul zice „numere întregi”. În `victor-vectori` ai avut de mai multe ori `% 2 == 1`, care ratează `-3` (în C++, `-3 % 2` e `-1`). Aici ai scris forma care merge și pe negative, fără să ți-o fi cerut nimeni.

varianta83 — `abs(v[i])` înainte de a lua cifrele, apoi suma pe `v[i]` original. Ai separat corect „numărul pe care îl analizez” de „numărul pe care îl adun”. Pe `-212` merge.

varianta91 — garda `if (n == 0) return 1;` e cea care salvează răspunsul. Pe `570`, apelul `par(570 % 10)` e `par(0)`. Fără garda ta, bucla `while (n != 0)` nu s-ar executa deloc și `par(0)` ar întoarce `0` — adică „0 nu e cifră pară”. Răspunsul ar fi fost `1` în loc de `2`. Ai acoperit un caz pe care exemplul din enunț îl conține, dar pe care nu-l spune.

varianta86 — cele mai mari două numere într-o singură parcurgere, cu `max2 = max` mutat înainte de `max = v[i]`. Ordinea celor două atribuiri e exact lucrul care se greșește, și tu ai pus-o bine. Și e „eficient din punct de vedere al timpului și al memoriei”, cum cerea enunțul: fără tablou, fără sortare.

Și sub din varianta8: corect pe toate cele 7 cazuri, inclusiv `a={20,22} b={1,21,23} → 1 20 21 22 23`, unde trebuie să sari peste elemente din **ambele** tablouri. Mecanismul din fișa „doi indici” e acum în fișierul unde trebuia, sub antetul cerut. Asta a durat 4 runde și s-a închis complet.

Q&A — răspunde-mi la astea trei

1. `varianta3.h` . Fără să rulezi: ce afișează programul tău pentru `n = 11111111` , și **de ce exemplul din enunț nu putea, oricât l-ai fi rulat, să-ți arate greșeala?** Ce proprietate are `21523531` care ascunde bug-ul?
2. `varianta80.h` . Urmărește `vf` pe `{30,25,12}` , pas cu pas, prin toată bucla. **Ce valoare are la final, și la ce pas a încetat să mai fie `-1` ?** Apoi: în fix, ce înlocuiește steagul, și de ce nu mai are nevoie de el?
3. `varianta73.h` . Când bucla `while (inf <= sup)` se oprește, `inf` e cu 1 mai mare decât `sup` . **Ce știi despre `x[sup]` și despre `x[inf]` față de `v` în acel moment?** De aici, de ce e `inf` poziția de inserare?

Ce ai de făcut, în ordine

Prima dată, o regulă nouă — înainte de orice commit: pentru fiecare variantă, pe lângă exemplul din enunț, **inventează un exemplu al tău** care încearcă să strice programul. Trei întrebări care găsesc aproape tot:

- Ce se întâmplă dacă **nu există** niciun element bun? (`varianta80`)
- Ce se întâmplă dacă un element se **repetă de mai multe ori** decât în exemplu? (`varianta3`)
- Ce se întâmplă **la capete** — cel mai mic, cel mai mare? (`varianta73`)

Apoi, într-o linie fiecare:

1. `varianta24.h:78` — `cout << fixed << setprecision(3) << medie;` , și `;` șters de pe linia 5 (M1)
2. `varianta92.h:45` — linia ștersă (M6)
3. `varianta67.h:26` — `d * d <= aux` (M5)
4. `varianta8.h` — `solutie4` cu doar `sub(a, 5, b, 3);` (M2)
5. `varianta23.h:31` — `i < n - 1` (C1, a treia rundă)

Apoi cele trei 🚫 , cu codul din FIX:

6. `varianta3.h` — B1, `if` -ul devine `for` până la `f[i] / 2`
7. `varianta73.h` — B2, `sup = n` și `return inf;` · apoi scrie b) și c)
8. `varianta80.h` — B3, tabloul `b`, sortarea, `k == 0`

O dată pentru tot repo-ul:

9. M7 — `namespace vNN { }` în fiecare header.
10. M4 — citirea din fișier, începând cu `varianta80.h` .

Runda 3 — 2026-09-29 — sha `dad5251`

Un commit (`dad5251`), 13 fişiere, 511 linii. Poarta de compilare: trecută, 32/32. Fişa `exercitii-doi-indici` rezolvată integral, 19/20 exemple corecte, iar mecanismul a migrat înapoi în `varianta8.h` .

Tiparul rundei: **review-ul devine comentariu** — fix-ul copiat ca `//!` în `varianta24.h` , codul neschimbat.

#	constatare	fișier	stare la runda 4
B1	fix-ul din runda 2 scris ca comentariu	varianta24.h	🟡 pe jumătate — 5.667 ✅ , mini / maxi încă afișate → M1 runda 4
B2	doarInA fără NU EXISTA	tema...doi- indici.h:172	✅ închis
B3	palindromul nu se construiește	varianta3.h:70	🔴 rescris, tot greșit pe cifre repetate → B1 runda 4
B4	sub , cerut de enunț, nu există	varianta8.h	✅ închis — corect pe 7/7 · neapelat → M2 runda 4
B5	alklea fără return	tema...doi- indici.h:423	✅ închis
M1	proiectul compilează 1 fișier din 32	varianteBac.cpp	🔴 agravat, 1 din 46 → M3 runda 4
M2	reuniune afișează și nu resetează p	tema...doi-indici.h	🔴 neatins
M3	zero fstream	tot repo-ul	🔴 neatins → M4 runda 4
M4	p / sortare distrug tabloul apelantului	v42, v8	🔴 neatins
M5	18× solutie3	tot repo-ul	🔴 agravat → M7 runda 4
M6	prompturi la tastatură	v1, v6, v11	🔴 neatins
M7	k necitit	varianta46.h	🔴 neatins

#	constatare	fișier	stare la runda 4
M8	<code>int n;</code> necitit	<code>varianta55.h:33</code>	● neatins
C1	<code>shift</code> citește <code>x[n]</code>	<code>varianta23.h:31</code>	● neatins → C1 runda 4
C2–C6	variabile nefolosite, marcaje, nume	diverse	● neatinse

Runda 2 — 2026-09-23 — sha 02640ce

Patru commit-uri (a9c3abc , d068314 , 7bbcd5c , 02640ce). 16 variante noi, 5 atinse. Poarta de compilare: trecută, 30/30 headere, warning-uri în 4 fișiere din care 3 bug-uri reale.

Verdict: algoritmii s-au făcut buni — 13 din 16 variante noi dau răspunsul corect. Tiparul runde: *rezolvă altă problemă decât cea cerută* — subprogramul cerut ocolit, datele nu vin din sursa indicată, formatul de ieșire ignorat. Vechiul tipar („te oprești la un pas de final”) s-a concentrat în trei `if`-uri cu corpul gol.

#	constatare	fișier	stare la runda 3
B1	nu afișează nimic; cif definit și apelat de 0 ori	varianta3.h	🟡 pe jumătate — cif apelat ✓, palindromul lipsă → B3 runda 3
B2	sub tot gol; interclasare pierde elemente pe a={2,4} b={1}	varianta8.h	🟡 pe jumătate — algorithm ✓ corect pe 4/4 , sub inexistent → B4 runda 3
B3	4, 8 ,5.66667 în loc de 5.667	varianta24.h	🔴 neatins — fix-ul scris ca comentariu → B1 runda 3
B4	mesajul nu exista numere prime	varianta5.h	✓ închis — NU EXISTA + citire de la tastatură
B5	f schelet: if gol, int k nu int &k , int fără return	varianta25.h	✓ închis — dă 24
B6	un 0 în plus la final; trei convenții de indexare	varianta26.h	✓ închis — 10 2 19 9 15
B7	i < d în loc de i <= d	varianta52.h	✓ închis — dă 4
M1	9 variante nu citesc din sursa cerută; zero fstream	6 enunțuri	🔴 neatins → M3 runda 3
M2	coliziuni de nume (17× solutie3)	tot repo-ul	🔴 agravat (18×) → M5 runda 3
M3	p sortează in-place tabloul apelantului	varianta42.h	🔴 neatins → M4 runda 3

#	constatare	fișier	stare la runda 3
M4	<code>k</code> nu se citește, tabloul nu se construiește	<code>varianta46.h</code>	🔴 neatins → M7 runda 3
M5	<code>ordin(0)</code> dă 1	<code>varianta55.h</code>	✅ închis — garda pusă · ⚠️ exemplul meu (<code>5</code> → <code>50</code>) era greșit, vezi runda 3
M6	prompturi la tastatură	6 locuri	🔴 neatins → M6 runda 3
M7	<code>nule</code> afișează, deși n-ar trebui	<code>varianta20.h</code>	✅ închis — afișarea mutată în <code>solutie3</code>
C1	<code>shift</code> citește <code>x[n]</code> , în afara zonei permutate	<code>varianta23.h:31</code>	🔴 neatins → C1 runda 3
C2	<code>count</code> peste <code>std::count</code>	<code>varianta18.h:18</code>	🔴 neatins
C3	variabile declarate și nefolosite, 5 locuri	v24, v3, v25, v8	🟡 4 din 5 dispărute cu rescrierile; rămâne <code>ct</code>
C4	<code>int f(long a)</code> — îngustare tăcută	<code>varianta5.h:27</code>	🔴 neatins
C5	<code><cmath></code> nefolosit în toate fișierele; <code>Header.h</code> orfan	tot repo-ul	🔴 neatins
C6	marcaje <code>///??</code> rămase	5 fișiere	🟡 2 din 5 sunt acum pe fișiere corecte → C4 runda 3
C7	spațiu în plus la final de linie	9 fișiere	🔴 neatins

#	constatare	fișier	stare la runda 3
C8	stergere citește x[dim]	varianta24.h:49	● neatins
C9	a[i] % k cu k = 0; fr[cif]++ fără gardă	v16, v14	● neatins

Trei ● închise curat, două pe jumătate. Cele închise sunt exact cele unde primise linia de cod; cele pe jumătate sunt acolo unde constatarea avea două lucruri în ea.

Corecte la runda 2, spuse pe nume: inversarea prin apeluri repetate de `shift` (`varianta23`), Euclid în lanț (`varianta66`), `ordin()` izolat ca subprogram (`varianta55`), două contoare cu ritmuri diferite în `fibo` (`varianta62`).

Livrate în aceeași rundă: `exercitii-runda-2.pdf` (8 exerciții, câte unul per mecanism greșit) și `exercitii-doi-indici.pdf` (7 exerciții pe mecanismul din `varianta8.h`, cu constrângerea „o singură parcurgere”). Pe a doua a rezolvat-o integral — vezi runda 3.

Runda 1 — 2026-09-15 — sha 46a53db

Commit 46a53db tema (variantele 3, 5, 24, 39) plus verificarea integrală a celor din 4281854 1st push . Poarta de compilare: trecută, 14/14 headere, 4 warning-uri din care 3 bug-uri reale.

Verdict: din cele 4 fișiere ale temei, unul singur (varianta39) producea răspunsul cerut. Tiparul rundeii: *te oprești la un pas de final* — algoritmul scris, linia care îl leagă de răspuns lipsă.

#	constatare	fișier	stare la runda 2
B1	parametrii de ieșire transmiși prin copie, nu prin referință	<code>varianta24.h:18</code>	✅ închis
B2	<code>for (int i = i; ...)</code> — bucla pornește de la o valoare nedefinită	<code>varianta24.h:23</code>	✅ închis
B3	nu afișează nimic; media s-ar calcula în întregi	<code>varianta24.h:57</code>	🟡 pe jumătate → B3 runda 2
B4	cifra 9 nu e numărată niciodată (<code>i < 9</code> pe un tablou de 10)	<code>varianta3.h:48,62</code>	⬜ dispărut odată cu rescrierea
B5	palindrom: <code>0</code> nu se afișează, zeroul lipsește, mijlocul lipsește	<code>varianta3.h:50-66</code>	🔴 înrăutățit → B1 runda 2
B6	punctul b) nu e scris: <code>f</code> nu e apelat niciodată; mesaj greșit	<code>varianta5.h:44</code>	🟡 pe jumătate → B4 runda 2
B7	funcție <code>int</code> care nu întoarce nimic	<code>varianta39.h:15</code>	✅ închis
B8	<code>sub</code> e schelet: interclasează și nu afișează nimic	<code>varianta8.h:18-33</code>	🔴 neatins → B2 runda 2
M1	două variante nu pot exista în același proiect (nume duplicate)	tot repo-ul	🔴 agravat → M2 runda 2

#	constatare	fișier	stare la runda 2
M2	<code>sort</code> definit de două ori și suprapus peste <code>std::sort</code>	<code>varianta1.h:44</code> , <code>varianta12.h:28</code>	● neatins
M3	prompturile de la tastatură strică evaluarea automată	5 locuri	● neatins → M6 runda 2
M4	<code>varianta39</code> cere citire din <code>bac.in</code> , nu din <code>cin</code>	<code>varianta39.h:21</code>	● neatins → M1 runda 2
M5	subprogramul <code>nule</code> afișează, deși n-ar trebui	<code>varianta20.h:29–31</code>	● neatins → M7 runda 2
C1	<code>a[i] % k</code> cu <code>k = 0</code>	<code>varianta16.h:19</code>	● neatins
C2	<code>fr[cif]++</code> fără gardă pe interval	<code>varianta14.h:22</code>	● neatins
C3	<code>int f(long a)</code> — îngustare tăcută pe <code>return a</code>	<code>varianta5.h:32</code>	● neatins → C4 runda 2
C4	<code><cmath></code> nefolosit peste tot; <code>Header.h</code> orfan	tot repo-ul	● neatins → C5 runda 2
C5	marcaje <code>///?</code> rămase	5 fișiere	● neatins → C6 runda 2

Corecte la runda 1, verificate prin rulare: `varianta1.h` (b), `varianta7.h` , `varianta12.h` , `varianta16.h` , `varianta17.h` , `varianta20.h` , `varianta39.h` , plus `varianta6.h` , `varianta11.h` , `varianta14.h` fără obiecții de fond.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro